بسم الله الرحمن الرحیم

دانشگاه بین‌المللی امام خمینی (ره)

دانشکدهٔ فنی و مهندسی — گروه مهندسی برق

گزارش کار آزمایشگاه — شمارهٔ ۱

آشنایی با میکروکنترلر آردوینو و پیاده‌سازی مدارهای دیجیتال روی بردبورد

شبیه‌سازی و اجرای پروژه‌های LED، دکمه، سنسور نور و دما در محیط Tinkercad Circuits

نام دانشجو	**مهدیه غلامی**
شمارهٔ دانشجویی	**۴۰۱۶۱۲۳۰۵۸**
نام استاد	**جناب آقای مهندس سلیمانی الموتی**
گروه	**۶۰**
درس	**آزمایشگاه سیستم‌های دیجیتال**
تاریخ	**۷ آبان ۱۴۰۵**

۷ آبان ۱۴۰۵

چکیده

در این گزارش، آشنایی با میکروکنترلر آردوینو و پیاده‌سازی مدارهای دیجیتال ساده بر روی بردبورد بررسی شده است. ابتدا ساختار محیط توسعهٔ آردوینو شامل دو تابع اصلی `setup()` و `loop()` و فرمان‌های پایه مانند `digitalWrite` ، `pinMode` و `digitalRead` معرفی می‌شود. سپس دوازده مسئلهٔ عملی شامل کنترل LED، کار با دکمه، LED چند رنگی (RGB)، سنسور نور (LDR) و سنسور دما طراحی و در محیط شبیه‌سازی Tinkercad Circuits پیاده‌سازی گردیده‌اند. نتایج به‌دست‌آمده نشان می‌دهد که تمامی مدارها مطابق با رفتار مورد انتظار و جدول صحت هر مسئله عمل می‌کنند و آردوینو به‌خوبی به‌عنوان کنترل‌کنندهٔ مرکزی مدارهای دیجیتال قابل استفاده است.

کلیدواژه‌ها: آردوینو، بردبورد، LED، دکمهٔ فشاری، سنسور نور LDR، سنسور دما، Tinkercad، مدار دیجیتال

فهرست مطالب

۱ مقدمه
۲ هدف آزمایش
۳ مبانی و مفاهیم برنامه‌نویسی آردوینو
۴ معرفی برد آردوینو و بردبورد
۵ شرح قسمت عملی آزمایش
۶ حل المسائل (پروژه‌های ۱ تا ۱۲)
۷ نتایج شبیه‌سازی
۸ تحلیل نتایج
۹ نتیجه‌گیری
۱۰ پیشنهادات
۱۱ منابع

۱. مقدمه

امروزه میکروکنترلرها مغز متفکر بیشتر دستگاه‌های الکترونیکی هوشمند هستند. آردوینو به‌عنوان یکی از محبوب‌ترین پلتفرم‌های متن‌باز، امکان ساخت و کنترل مدارهای دیجیتال را بدون پیچیدگی‌های سخت‌افزاری فراهم می‌کند. در این آزمایش، با نحوهٔ کار با برد آردوینو، اتصال قطعات روی بردبورد و نوشتن برنامه‌های کنترلی آشنا می‌شویم.

مدارهای دیجیتال بر پایهٔ منطق صفر و یک کار می‌کنند. با ترکیب ورودی‌ها (مانند دکمه و سنسور) و خروجی‌ها (مانند LED و بازر) می‌توان رفتارهای منطقی متنوعی را پیاده‌سازی کرد. آردوینو با ارائهٔ پین‌های ورودی/خروجی دیجیتال و آنالوگ، بستری ایده‌آل برای درک این مفاهیم در اختیار می‌گذارد.

۲. هدف آزمایش

۱ آشنایی با محیط توسعهٔ آردوینو و ساختار توابع `setup` و `loop`.

۲ یادگیری فرمان‌های پایهٔ `digitalWrite` ، `pinMode` و `digitalRead`.

۳ کار با بردبورد و اتصال صحیح قطعات بدون لحیم‌کاری.

۴ کنترل LED، دکمه، RGB و سنسورها و استفاده از متغیرها و پورت سریال.

💡 نکته: در زبان برنامه‌نویسی آردوینو، نوشتار حروف بزرگ و کوچک حساسیت دارد؛ بنابراین دستورات باید دقیقاً به‌صورت صحیح نوشته شوند.

۳. مبانی و مفاهیم برنامه‌نویسی آردوینو

void setup()

این بخش فقط یک‌بار در ابتدای اجرای برنامه اجرا می‌شود و برای پیکربندی اولیهٔ پین‌ها و شروع ارتباط سریال به کار می‌رود.

void loop()

دستورات این بخش به‌صورت مداوم و بدون توقف تکرار می‌شوند؛ منطق اصلی برنامه در اینجا نوشته می‌شود.

pinMode(pin, mode)

نوع کارکرد یک پین را مشخص می‌کند: `OUTPUT` برای خروجی و `INPUT` برای ورودی.

digitalWrite / digitalRead

فرمان اول وضعیت پین خروجی را روی `HIGH` (۵ ولت) یا `LOW` (صفر ولت) می‌برد و فرمان دوم وضعیت پین ورودی را می‌خواند.

استفاده از متغیرها به‌جای مقادیر ثابت

در پروژه‌های بزرگ‌تر، به‌جای نوشتن مقادیر ثابت (مانند شمارهٔ پین یا زمان تأخیر) از متغیرها استفاده می‌کنیم؛ در این صورت تنها با تغییر مقدار متغیر، کل مدار به‌روزرسانی می‌شود. رعایت اصول نام‌گذاری مانند CamelCase و انتخاب نام‌های معنادار (مانند `LEDpin` و `waitTime`) خوانایی کد را افزایش می‌دهد.

```
int LEDpin = 13;      // نام معنادار با CamelCase
int waitTime = 1000;  // زمان تأخیر بر حسب میلی‌ثانیه
```

۴. معرفی برد آردوینو و بردبورد

در برد آردوینو Uno، پین‌های ۰ تا ۱۳ برای ورودی و خروجی دیجیتال قابل استفاده هستند و پین ۱۳ به یک LED داخلی مجهز است. علاوه بر این، پین‌های آنالوگ A0 تا A5 برای خواندن سنسورها در دسترس‌اند.

پین‌های دیجیتال

پین‌های ۰ تا ۱۳؛ پین‌های دارای علامت PWM (~) قابلیت خروجی آنالوگ شبیه‌سازی‌شده را دارند.

پین‌های آنالوگ

پین‌های A0 تا A5 برای خواندن مقادیر سنسورها مانند نور و دما به کار می‌روند.

سوراخ‌های وسطی بردبورد

به‌صورت ستونی به هم متصل‌اند و برای قرار دادن پایه‌های قطعات استفاده می‌شوند.

ریل‌های کناری (VCC/GND)

به‌صورت افقی متصل‌اند و برای اتصال منابع تغذیه و زمین به کار می‌روند.

۵. شرح قسمت عملی آزمایش

به دلیل در دسترس نبودن فیزیکی برد، تمامی مدارها در محیط شبیه‌سازی آنلاین **Tinkercad Circuits** طراحی و اجرا شده‌اند. روش کار در هر مسئله به شرح زیر است:

1. قرار دادن برد آردوینو Uno و بردبورد در محیط شبیه‌سازی.
2. افزودن قطعات (LED، مقاومت، دکمه، سنسور) و سیم‌کشی مطابق صورت مسئله.
3. نوشتن کد در ویرایشگر و تنظیم پین‌ها در تابع `setup` .
4. اجرای شبیه‌سازی با دکمهٔ Start و مشاهدهٔ رفتار مدار و Serial Monitor.

⚡ نکتهٔ ایمنی: پایهٔ بلند LED (آند) به سمت ولتاژ مثبت و پایهٔ کوتاه (کاتد) به سمت زمین متصل می‌شود و همیشه یک مقاومت محدودکنندهٔ جریان (معمولاً ۲۲۰ اهم) سری با LED قرار می‌گیرد.

مسئلهٔ ۱ روشن و خاموش کردن LED با تأخیر یک‌ثانیه‌ای

صورت مسئله

یک مدار ساده طراحی کنید که یک LED را به پین ۱۳ آردوینو متصل کرده و با استفاده از کد آردوینو آن را به‌صورت متناوب روشن و خاموش کند؛ زمان هر حالت یک ثانیه باشد.

نکات کلیدی

- در تابع setup پین ۱۳ با فرمان pinMode به‌عنوان خروجی پیکربندی می‌شود.
- دستور digitalWrite با مقدار HIGH ولتاژ ۵ ولت و با LOW صفر ولت تولید می‌کند.
- تابع delay(1000) دقیقاً یک ثانیه مکث ایجاد می‌کند و این چرخه در loop تکرار می‌شود.

کد آردوینو

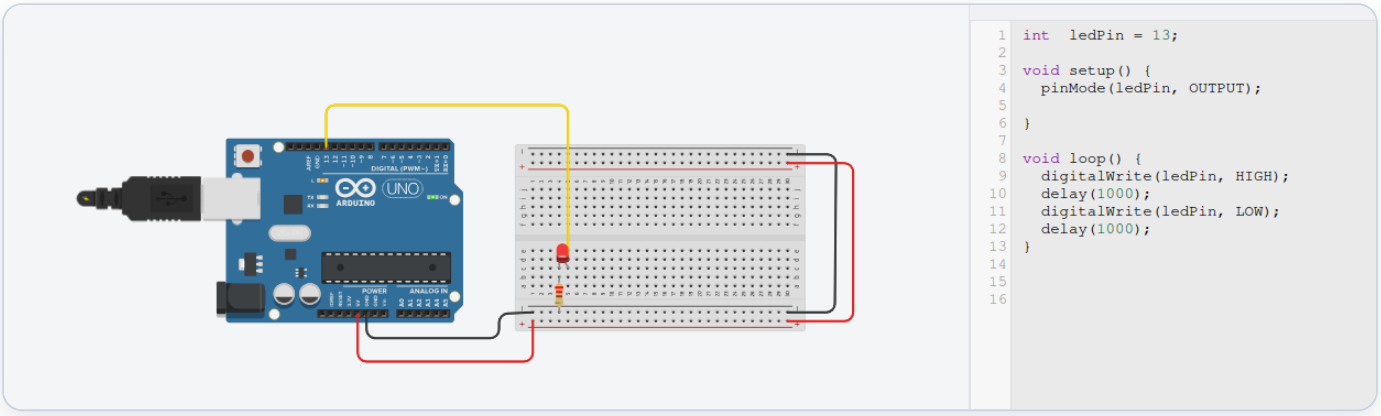
```
int ledPin = 13;

void setup() {
  pinMode(ledPin, OUTPUT);
}

void loop() {
  digitalWrite(ledPin, HIGH);
  delay(1000);
  digitalWrite(ledPin, LOW);
  delay(1000);
}
```

توضیح عملکرد

ساده‌ترین الگوی کنترل خروجی دیجیتال در این مدار پیاده‌سازی شده است. پین ۱۳ که روی برد آردوینو به یک LED داخلی متصل است، در تابع setup با فرمان pinMode به‌عنوان خروجی پیکربندی می‌شود. سپس در حلقهٔ loop، پین با digitalWrite در سطح HIGH قرار گرفته و LED روشن می‌شود؛ پس از یک ثانیه مکث، همان پین روی LOW رفته و LED خاموش می‌گردد. چون این دستورهای چهارگانه به‌صورت پی‌درپی و بی‌پایان تکرار می‌شوند، LED هر ثانیه یک‌بار چشمک می‌زند. این پروژه نقطهٔ شروع درک چرخهٔ اجرای برنامهٔ آردوینو و مفهوم سطح منطقی ولتاژ است.



شکل ۱: محیط شبیه‌سازی Tinkercad Circuits — چیدمان بردبورد، سیم‌کشی و کد اجراشده

مسئلهٔ ۲ تغییر زمان روشن و خاموش بودن LED از طریق پورت سریال

صورت مسئله

مداری طراحی کنید که یک LED را به پین ۹ آردوینو متصل کرده و زمان روشن و خاموش شدن آن را به‌صورت متغیر، از طریق ورودی سریال (Serial) تغییر دهد.

کد آردوینو

```
const int LEDpin = 9;
int onTime = 1000;
int offTime = 1000;

void setup() {
  pinMode(LEDpin, OUTPUT);
  Serial.begin(9600);
  Serial.println("Enter ON time and OFF time in ms:");
  Serial.println("Example: 500 1000");
}

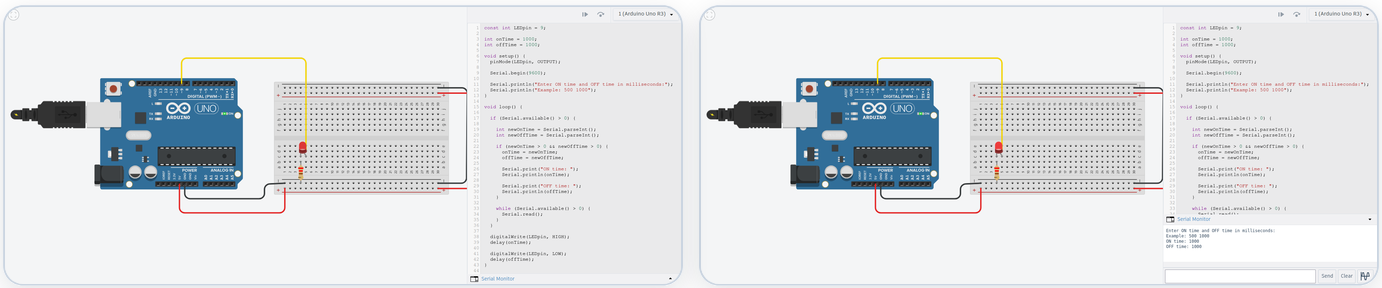
void loop() {
  if (Serial.available() > 0) {
    int newOnTime = Serial.parseInt();
    int newOffTime = Serial.parseInt();
    if (newOnTime > 0 && newOffTime > 0) {
      onTime = newOnTime;
      offTime = newOffTime;
      Serial.print("ON time: ");
      Serial.println(onTime);
      Serial.print("OFF time: ");
      Serial.println(offTime);
    }
    while (Serial.available() > 0) {
      Serial.read();
    }
  }
  digitalWrite(LEDpin, HIGH);
  delay(onTime);
  digitalWrite(LEDpin, LOW);
  delay(offTime);
}
```

نکات کلیدی

- با Serial.available() بررسی می‌شود که داده‌ای از مانیتور سریال ارسال شده است یا خیر.
- فرمان Serial.parseInt() دو عدد ورودی را به‌عنوان زمان روشن و خاموش شدن می‌خواند.
- حلقهٔ while باقیماندهٔ بافر سریال را پاک‌سازی می‌کند تا خواندن بعدی درست انجام شود.

توضیح عملکرد

در این مسئله، زمان‌های روشن و خاموش بودن LED از پیش ثابت نیستند، بلکه در زمان اجرا و از طریق پورت سریال قابل تغییرند. برد در تابع setup ارتباط سریال را با نرخ ۹۶۰۰ باز می‌کند و از کاربر می‌خواهد دو عدد (زمان روشن و زمان خاموش، بر حسب میلی‌ثانیه) وارد کند. در هر دور از loop ابتدا با Serial.available بررسی می‌شود که آیا داده‌ای در بافر موجود است؛ در صورت مثبت بودن، با دو فراخوانی Serial.parseInt آن دو عدد خوانده می‌شوند و پس از اطمینان از بزرگ‌تر بودنشان از صفر، در متغیرهای onTime و offTime ذخیره می‌گردند. سپس LED دقیقاً با همین مقادیر تازه چشمک می‌زند. حلقهٔ while نیز باقیماندهٔ بافر را پاک می‌کند تا خواندن بعدی با خطا انجام نشود.



شکل ۲: محیط شبیه‌سازی Tinkercad Circuits — چیدمان بردبورد، سیم‌کشی و کد اجراشده

مسئلهٔ ۳ کنترل LED با استفاده از یک دکمه

صورت مسئله

یک دکمه به پین ۲ آردوینو متصل کنید؛ به‌گونه‌ای که با فشار دادن دکمه، LED روشن و با رها کردن آن، LED خاموش شود.

کد آردوینو

```
int ledPin = 13;
int buttonPin = 2;
int buttonState = 0;

void setup() {
  pinMode(ledPin, OUTPUT);
  pinMode(buttonPin, INPUT);
}

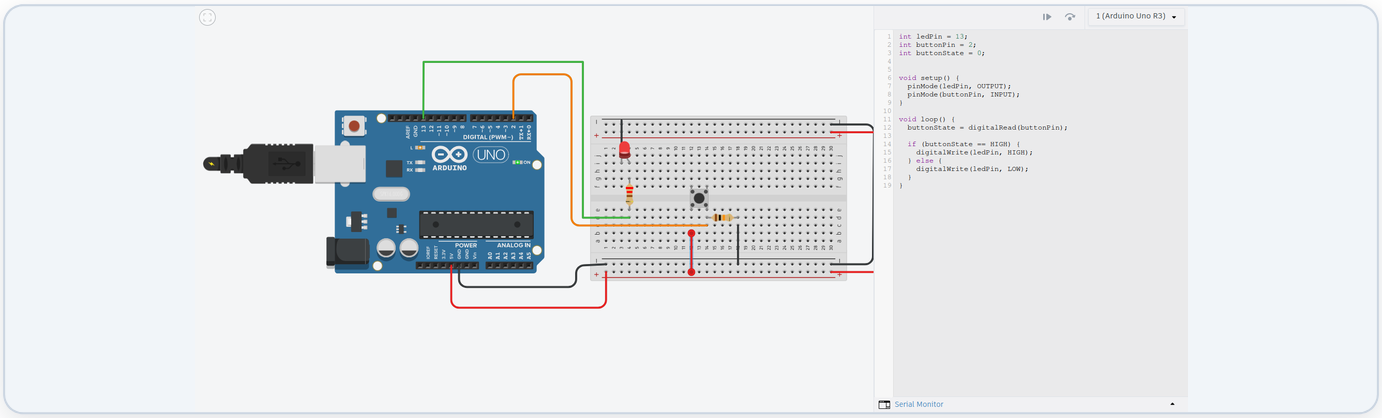
void loop() {
  buttonState = digitalRead(buttonPin);
  if (buttonState == HIGH) {
    digitalWrite(ledPin, HIGH);
  } else {
    digitalWrite(ledPin, LOW);
  }
}
```

توضیح عملکرد

این مدار یک کلید کنترلی ساده می‌سازد؛ دکمه به پین ۲ (ورودی) و LED به پین ۱۳ (خروجی) متصل شده‌اند. در هر دور از loop، وضعیت دکمه با digitalRead خوانده و در متغیر buttonState ذخیره می‌شود. اگر دکمه فشرده باشد و ورودی در سطح HIGH قرار گیرد، LED روشن می‌شود و در غیر این صورت خاموش می‌ماند. بنابراین رفتار LED آینه‌وارِ وضعیت دکمه است؛ به‌محض فشار دادن روشن و به‌محض رها کردن خاموش می‌گردد. در عمل برای اطمینان از سطح منطقی مشخص در حالت آزاد، یک مقاومت پول‌داون روی خط دکمه در نظر گرفته می‌شود.

نکات کلیدی

- دستور digitalRead وضعیت منطقی پین دکمه را می‌خواند و در متغیر buttonState ذخیره می‌کند.
- در این پیاده‌سازی مستقیم، LED آینه‌وار وضعیت دکمه را نشان می‌دهد.
- برای اطمینان از رفتار صحیح، معمولاً یک مقاومت پول‌داون روی خط دکمه در نظر گرفته می‌شود.



شکل ۳: محیط شبیه‌سازی Tinkercad Circuits — چیدمان بردبورد، سیم‌کشی و کد اجراشده

مسئلهٔ ۴ نمایش وضعیت با دو LED و یک دکمه (Toggle)

صورت مسئله

مداری بسازید که از دو LED استفاده کند؛ یکی نشانهٔ «روشن» و دیگری نشانهٔ «خاموش». هر بار که دکمه فشرده می‌شود، وضعیت بین دو حالت جابه‌جا شود.

کد آردوینو

```
const int redLED = 2;
const int greenLED = 3;
const int buttonPin = 4;
int state = 0;
int lastButtonState = HIGH;

void setup() {
  pinMode(redLED, OUTPUT);
  pinMode(greenLED, OUTPUT);
  pinMode(buttonPin, INPUT_PULLUP);
  digitalWrite(redLED, LOW);
  digitalWrite(greenLED, HIGH);
}

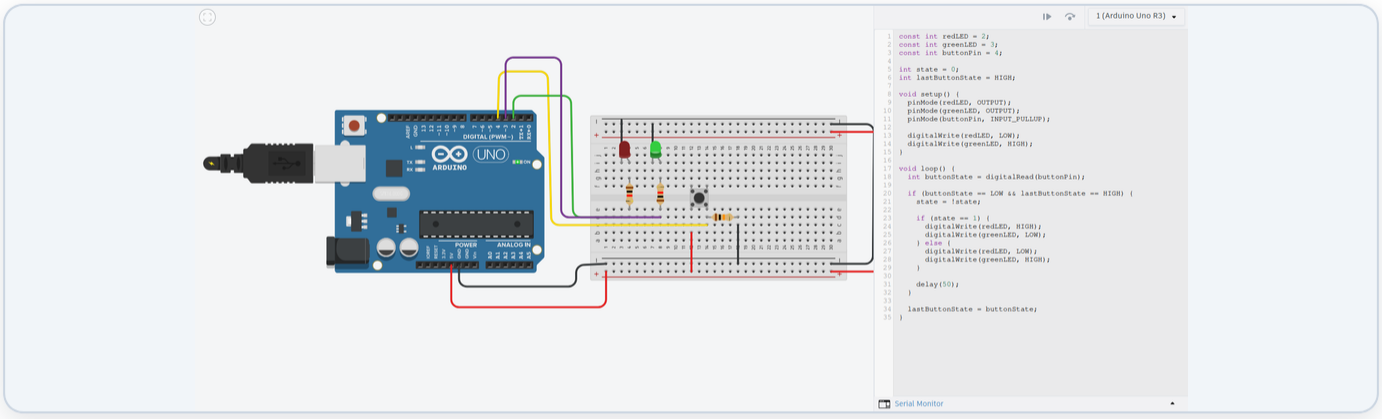
void loop() {
  int buttonState = digitalRead(buttonPin);
  if (buttonState == LOW && lastButtonState == HIGH) {
    state = !state;
    if (state == 1) {
      digitalWrite(redLED, HIGH);
      digitalWrite(greenLED, LOW);
    } else {
      digitalWrite(redLED, LOW);
      digitalWrite(greenLED, HIGH);
    }
    delay(50);
  }
  lastButtonState = buttonState;
}
```

توضیح عملکرد

در این پروژه دکمه نقش یک کلید تعویض‌کننده (Toggle) را دارد: به‌جای آن‌که LED فقط هنگام فشار دادن روشن بماند، هر بار فشردن دکمه وضعیت را بین دو حالت جابه‌جا می‌کند. دو LED قرمز و سبز نشانهٔ دو وضعیت «روشن» و «خاموش» هستند. نکتهٔ کلیدی، تشخیص لحظهٔ فشار دادن (نه وضعیت فشرده‌بودن) است؛ به همین دلیل وضعیت فعلی دکمه با وضعیت پیشین (lastButtonState) مقایسه می‌شود و تنها در «لبهٔ» تغییر، متغیر state معکوس می‌گردد. با پیکربندی INPUT_PULLUP حالت عادی دکمه HIGH و حالت فشرده LOW است و تأخیر ۵۰ میلی‌ثانیه‌ای پرش‌های مکانیکی کلید را حذف می‌کند.

نکات کلیدی

- با INPUT_PULLUP، حالت عادی دکمه HIGH و حالت فشرده LOW است.
- شرط buttonState == LOW && lastButtonState == HIGH لبهٔ افت ولتاژ (فشار دکمه) را تشخیص می‌دهد.
- دستور delay(50) برای حذف پرش‌های مکانیکی دکمه (Debounce) به کار رفته است.



شکل ۴: محیط شبیه‌سازی Tinkercad Circuits — چیدمان بردبورد، سیم‌کشی و کد اجراشده

مسئلهٔ ۵ چشمک‌زن متناوب با دو LED

صورت مسئله

مداری بسازید که در آن دو LED به‌طور متناوب روشن و خاموش شوند (یکی روشن و دیگری خاموش و بالعکس)؛ مدت زمان روشن بودن هر LED یک ثانیه باشد.

کد آردوینو

```
int LED1 = 2;
int LED2 = 3;

void setup() {
  pinMode(LED1, OUTPUT);
  pinMode(LED2, OUTPUT);
}

void loop() {
  digitalWrite(LED1, HIGH);
  digitalWrite(LED2, LOW);
  delay(1000);

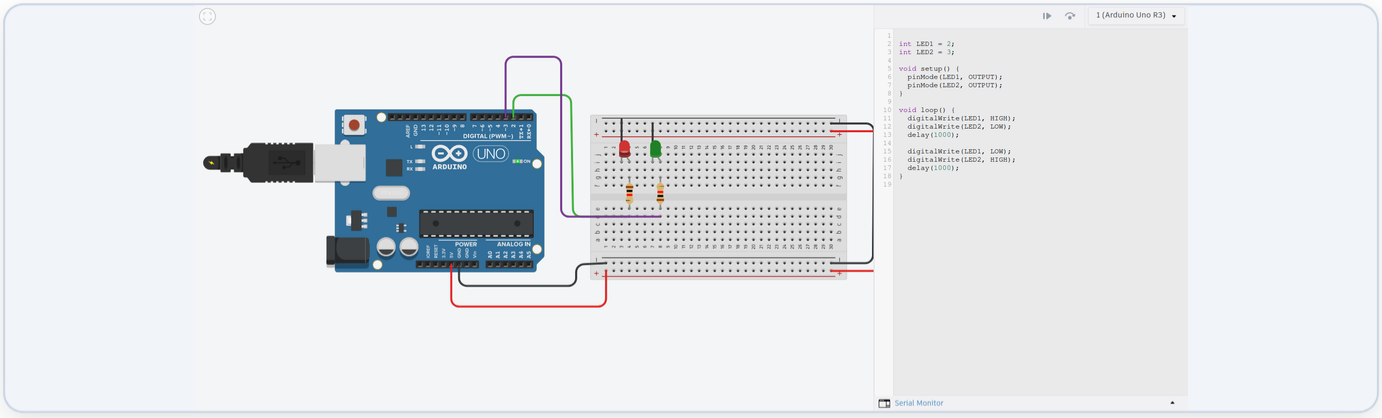
  digitalWrite(LED1, LOW);
  digitalWrite(LED2, HIGH);
  delay(1000);
}
```

نکات کلیدی

- در نیم‌چرخهٔ اول، LED1 روشن و LED2 خاموش می‌شود و پس از یک ثانیه حالت معکوس می‌گردد.
- این رفتار مکمل، پایهٔ ساخت نشانگرهای وضعیت و مدارهای چشمکی دو فاز است.

توضیح عملکرد

این مدار دو LED را به‌صورت مکمل و یک‌درمیان روشن می‌کند. در نیم‌چرخهٔ نخست LED1 روشن و LED2 خاموش می‌شود و به مدت یک ثانیه در همین حالت می‌ماند؛ سپس در نیم‌چرخهٔ دوم جای دو LED عوض می‌شود. چون در هر لحظه همیشه یکی روشن و دیگری خاموش است، الگوی چشمکی دوفازی ایجاد می‌شود که پایهٔ بسیاری از نشانگرها، تابلوها و مدارهای زمان‌بندی دیجیتال به شمار می‌رود.



شکل ۵: محیط شبیه‌سازی Tinkercad Circuits — چیدمان بردبورد، سیم‌کشی و کد اجراشده

مسئلهٔ ۶ محافظت از LED با مقاومت و زمان‌بندی دو ثانیه

صورت مسئله

مداری طراحی کنید که از یک LED و یک مقاومت ۲۲۰ اهم برای محافظت استفاده کند؛ کد به‌گونه‌ای باشد که LED به مدت دو ثانیه روشن و سپس خاموش شود.

نکات کلیدی

- مقاومت ۲۲۰ اهم جریان عبوری از LED را محدود کرده و از سوختن آن جلوگیری می‌کند.
- محاسبهٔ ساده: $R = (۵ - ۲)/۰٫۰۲ \approx ۱۵۰$ اهم؛ مقدار ۲۲۰ اهم حاشیهٔ اطمینان ایجاد می‌کند.

کد آردوینو

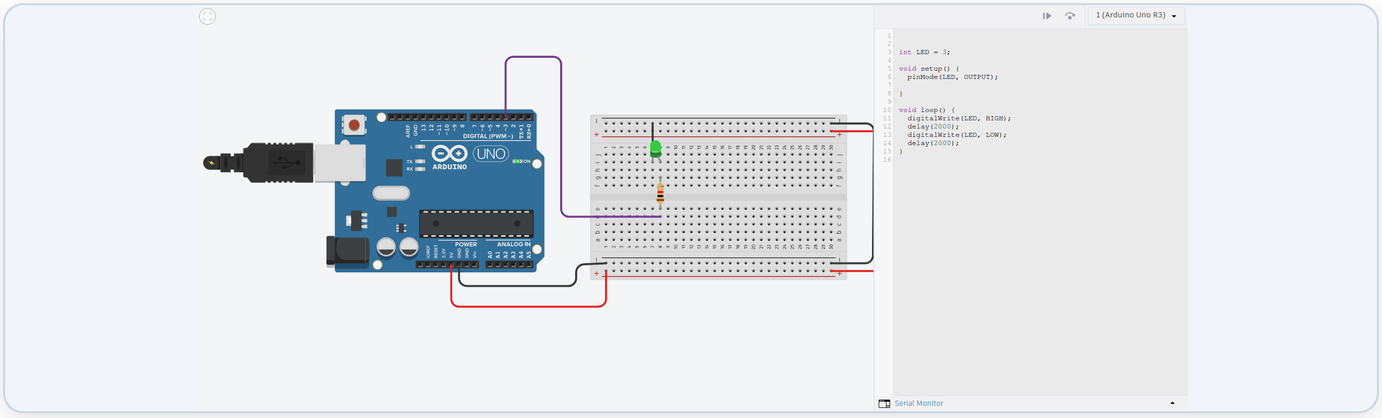
```
int LED = 3;

void setup() {
  pinMode(LED, OUTPUT);
}

void loop() {
  digitalWrite(LED, HIGH);
  delay(2000);
  digitalWrite(LED, LOW);
  delay(2000);
}
```

توضیح عملکرد

این مسئله دو هدف را هم‌زمان دنبال می‌کند: محافظت از LED و زمان‌بندی دقیق. LED از پین ۳ کنترل می‌شود و یک مقاومت ۲۲۰ اهم به‌صورت سری با آن قرار می‌گیرد تا جریان از حد مجاز (معمولاً حدود ۲۰ میلی‌آمپر) فراتر نرود و LED آسیب نبیند. در کد، LED به مدت دو ثانیه روشن و سپس دو ثانیه خاموش می‌ماند. مقدار مقاومت از قانون اهم به‌دست می‌آید: با ولتاژ تغذیهٔ ۵ ولت، افت ولتاژ LED حدود ۲ ولت و جریان ۲۰ میلی‌آمپر، مقاومت مطلوب نزدیک به ۱۵۰ اهم است و انتخاب ۲۲۰ اهم حاشیهٔ اطمینان ایجاد می‌کند.



شکل ۶: محیط شبیه‌سازی Tinkercad Circuits — چیدمان بردبورد، سیم‌کشی و کد اجراشده

مسئلهٔ ۷ کنترل LED چند رنگی (RGB)

صورت مسئله

یک LED RGB به آردوینو وصل کنید و با استفاده از پین‌های مناسب، رنگ‌های مختلف را کنترل نمایید؛ به‌ترتیب قرمز، سپس سبز و بعد آبی.

کد آردوینو

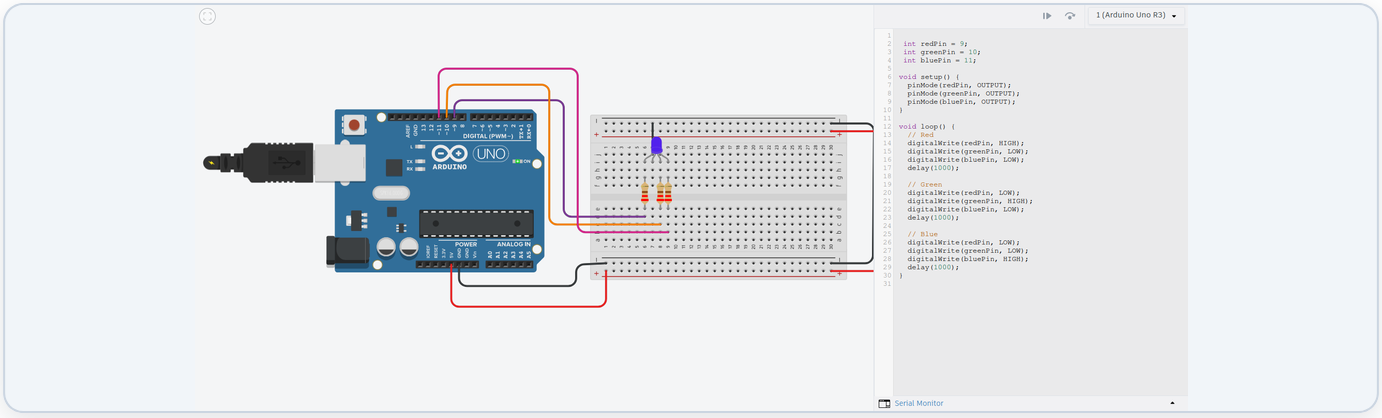
```arduino
int redPin = 9;
int greenPin = 10;
int bluePin = 11;

void setup() {
  pinMode(redPin, OUTPUT);
  pinMode(greenPin, OUTPUT);
  pinMode(bluePin, OUTPUT);
}

void loop() {
  // Red
  digitalWrite(redPin, HIGH);
  digitalWrite(greenPin, LOW);
  digitalWrite(bluePin, LOW);
  delay(1000);
  // Green
  digitalWrite(redPin, LOW);
  digitalWrite(greenPin, HIGH);
  digitalWrite(bluePin, LOW);
  delay(1000);
  // Blue
  digitalWrite(redPin, LOW);
  digitalWrite(greenPin, LOW);
  digitalWrite(bluePin, HIGH);
  delay(1000);
}
```

توضیح عملکرد

LED RGB از سه بخش نوردهندهٔ قرمز، سبز و آبی تشکیل شده که هر یک با یک پین جداگانه کنترل می‌شود. در این برنامه در هر مرحله فقط یکی از سه رنگ فعال می‌گردد: نخست قرمز، سپس سبز و در پایان آبی؛ هر رنگ به مدت یک ثانیه روشن می‌ماند. با توجه به اصل ترکیب جمعی (additive) رنگ‌ها، فعال‌کردن هم‌زمان دو یا سه پایه می‌تواند رنگ‌های ثانویه‌ای مانند زرد، فیروزه‌ای و ارغوانی را نیز تولید کند؛ بنابراین این مدار پایهٔ ساخت نمایشگرهای رنگی است.



شکل ۷: محیط شبیه‌سازی Tinkercad Circuits — چیدمان بردبورد، سیم‌کشی و کد اجراشده

نکات کلیدی

- هر رنگ LED RGB از یک کاتد مشترک و سه پایهٔ جداگانه تشکیل شده است.
- با فعال‌سازی هم‌زمان ترکیب این سه پایه می‌توان رنگ‌های ثانویه (زرد، فیروزه‌ای، ارغوانی) تولید کرد.

مسئلهٔ ۸ چشمک‌زن با دو LED RGB

صورت مسئله

مداری طراحی کنید که دو LED RGB به‌طور متناوب چشمک بزنند (یکی قرمز و دیگری سبز)؛ زمان روشن بودن هر LED به‌صورت جداگانه تنظیم شود.

نکات کلیدی

- با تفکیک زمان onTime1 و onTime2 می‌توان ریتم چشمک‌زن را به‌صورت مستقل تنظیم کرد.
- این الگو پایهٔ ساخت تابلوهای روان و نشانگرهای چند رنگ است.

کد آردوینو

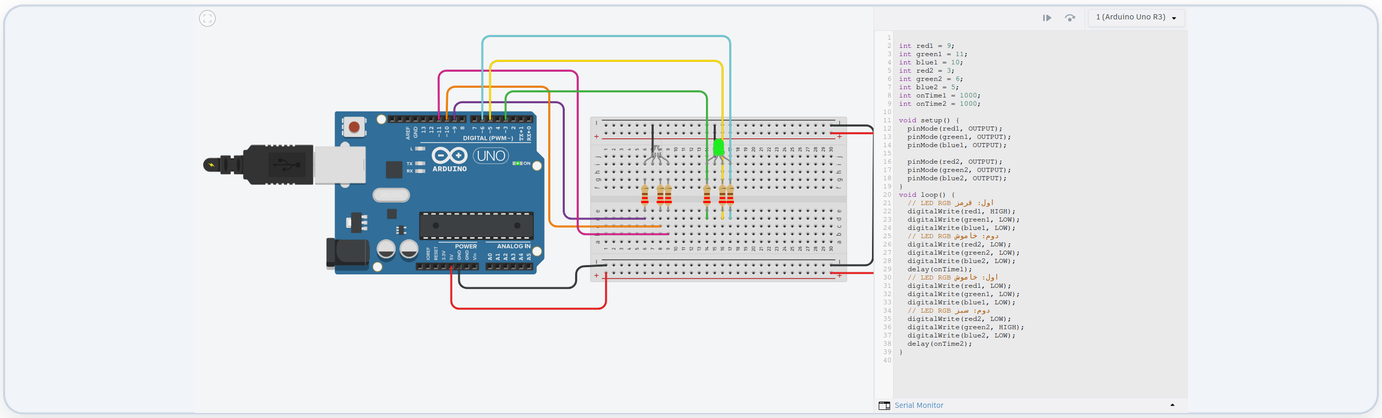
```
int red1 = 9, green1 = 11, blue1 = 10;
int red2 = 3, green2 = 6, blue2 = 5;
int onTime1 = 1000;
int onTime2 = 1000;

void setup() {
  pinMode(red1, OUTPUT);
  pinMode(green1, OUTPUT);
  pinMode(blue1, OUTPUT);
  pinMode(red2, OUTPUT);
  pinMode(green2, OUTPUT);
  pinMode(blue2, OUTPUT);
}

void loop() {
  // LED RGB اول: قرمز
  digitalWrite(red1, HIGH);
  digitalWrite(green1, LOW);
  digitalWrite(blue1, LOW);
  // LED RGB دوم: خاموش
  digitalWrite(red2, LOW);
  digitalWrite(green2, LOW);
  digitalWrite(blue2, LOW);
  delay(onTime1);
  // LED RGB اول: خاموش
  digitalWrite(red1, LOW);
  digitalWrite(green1, LOW);
  digitalWrite(blue1, LOW);
  // LED RGB دوم: سبز
  digitalWrite(red2, LOW);
  digitalWrite(green2, HIGH);
  digitalWrite(blue2, LOW);
  delay(onTime2);
}
```

توضیح عملکرد

این پروژه ترکیب دو مفهوم پیشین است: دو LED RGB که به‌صورت متناوب و مکمل چشمک می‌زنند. در گام نخست RGB اول روی قرمز روشن می‌شود در حالی‌که RGB دوم کاملاً خاموش است؛ پس از گذشت زمان onTime1 حالت برعکس می‌شود و این بار RGB دوم روی سبز روشن می‌گردد. با جدا در نظر گرفتن دو متغیر زمانی onTime1 و onTime2 می‌توان ریتم هر دو LED را به‌طور مستقل تنظیم کرد. این الگو در تابلوهای روان و سامانه‌های هشدار چندرنگ کاربرد فراوان دارد.



شکل ۸: محیط شبیه‌سازی Tinkercad Circuits — چیدمان بردبورد، سیم‌کشی و کد اجراشده

مسئلهٔ ۹ کنترل LED بر اساس سنسور دما

صورت مسئله

یک سنسور دما (مانند LM35/TMP36) به آردوینو وصل کنید؛ وقتی دمای محیط بالاتر از ۳۰ درجهٔ سلسیوس شد، LED قرمز روشن و در غیر این صورت LED سبز روشن باشد.

نکات کلیدی

- مبدل آنالوگ به دیجیتال آردوینو ۱۰ بیت است؛ بنابراین analogRead مقداری بین ۰ تا ۱۰۲۳ برمی‌گرداند.
- سنسورهای سری TMP/LM35 به ازای هر درجهٔ سلسیوس ۱۰ میلی‌ولت خروجی می‌دهند؛ پس temperature = voltage × ۱۰۰.
- شرط if دمای آستانهٔ ۳۰ درجه را به یک هشدار بصری (قرمز/سبز) تبدیل می‌کند.

کد آردوینو

```
int tempPin = A0;
int redLED = 8;
int greenLED = 9;
int sensorValue;
float voltage;
float temperatureC;

void setup() {
  pinMode(redLED, OUTPUT);
  pinMode(greenLED, OUTPUT);
  Serial.begin(9600);
}

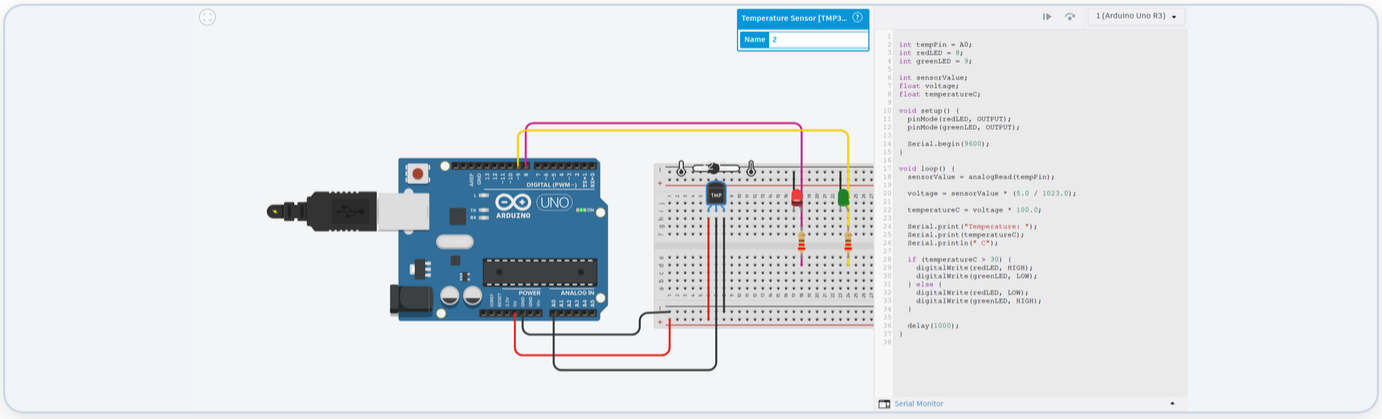
void loop() {
  sensorValue = analogRead(tempPin);
  voltage = sensorValue * (5.0 / 1023.0);
  temperatureC = voltage * 100.0;

  Serial.print("Temperature: ");
  Serial.print(temperatureC);
  Serial.println(" C");

  if (temperatureC > 30) {
    digitalWrite(redLED, HIGH);
    digitalWrite(greenLED, LOW);
  } else {
    digitalWrite(redLED, LOW);
    digitalWrite(greenLED, HIGH);
  }
  delay(1000);
}
```

توضیح عملکرد

در این مدار، سنسور دما نقش یک ورودی آنالوگ را بازی می‌کند. خروجی سنسور به پین A0 متصل است و مقدار آن با analogRead خوانده می‌شود؛ چون مبدل آنالوگ‌به‌دیجیتال آردوینو ۱۰ بیتی است، عددی بین ۰ تا ۱۰۲ به‌دست می‌آید. این عدد نخست با نسبت ۵/۱۰۲۳ به ولتاژ و سپس با ضریب ۱۰۰ به دما تبدیل می‌شود (در سنسورهای این سری هر درجه سلسیوس معادل ۱۰ میلی‌ولت است). اگر دما از آستانهٔ ۳۰ درجه بیشتر شود، LED قرمز به‌عنوان هشدار روشن و LED سبز خاموش می‌گردد؛ در غیر این صورت LED سبز روشن می‌ماند. مقدار دما نیز برای پایش روی Serial Monitor چاپ می‌شود.



شکل ۹: محیط شبیه‌سازی Tinkercad Circuits — چیدمان بردبورد، سیم‌کشی و کد اجراشده

مسئلهٔ ۱۰ کنترل LED با سنسور نور (LDR)

صورت مسئله

یک سنسور نور (LDR) به آردوینو وصل کنید؛ برنامه‌ای بنویسید که در صورت کم بودن نور محیط، LED روشن و در صورت زیاد بودن نور، LED خاموش شود.

نکات کلیدی

- سنسور LDR با یک مقاومت، یک تقسیم‌کنندهٔ ولتاژ می‌سازد که خروجی آن به پین آنالوگ A0 داده می‌شود.
- در تاریکی مقاومت LDR زیاد و مقدار خوانده‌شده تغییر می‌کند؛ با یک آستانه، LED روشن می‌شود.

کد آردوینو

```
int LDRpin = A0;
int LEDpin = 8;
int lightValue;

void setup() {
  pinMode(LEDpin, OUTPUT);
  Serial.begin(9600);
}

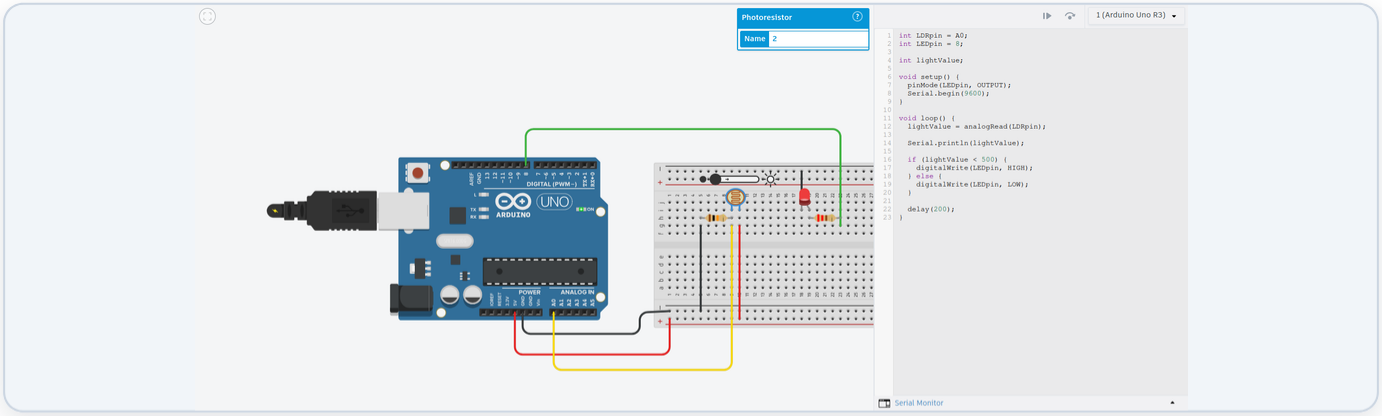
void loop() {
  lightValue = analogRead(LDRpin);

  Serial.println(lightValue);

  if (lightValue < 500) {
    digitalWrite(LEDpin, HIGH);
  } else {
    digitalWrite(LEDpin, LOW);
  }
  delay(200);
}
```

توضیح عملکرد

سنسور نور LDR مقاومتی متغیر دارد که با افزایش شدت نور کاهش می‌یابد. این سنسور به‌همراه یک مقاومت ثابت، یک تقسیم‌کنندهٔ ولتاژ می‌سازد که نقطهٔ میانی آن به پین آنالوگ A0 وصل است. در هر دور از loop، مقدار نور با analogRead خوانده و روی سریال نمایش داده می‌شود. اگر مقدار خوانده‌شده کمتر از آستانهٔ ۵۰۰ باشد (یعنی محیط نسبتاً تاریک باشد) LED روشن و در غیر این صورت خاموش می‌شود. به این ترتیب مدار مانند یک چراغ خودکار عمل می‌کند که در تاریکی روشن می‌شود.



شکل ۱۰: محیط شبیه‌سازی Tinkercad Circuits — چیدمان بردبورد، سیم‌کشی و کد اجراشده

مسئلهٔ ۱۱ چشمک‌زن با سرعت متغیر از طریق سریال

صورت مسئله

مداری بسازید که LED با سرعت متغیر (از ۵۰۰ میلی‌ثانیه تا ۳ ثانیه) چشمک بزند؛ برای تغییر سرعت، زمان تأخیر را از طریق ورودی سریال وارد کنید.

کد آردوینو

```arduino
int LEDpin = 9;
int blinkTime = 1000;

void setup() {
  pinMode(LEDpin, OUTPUT);
  Serial.begin(9600);
  Serial.println("Enter delay between 500 and 3000 ms:");
}

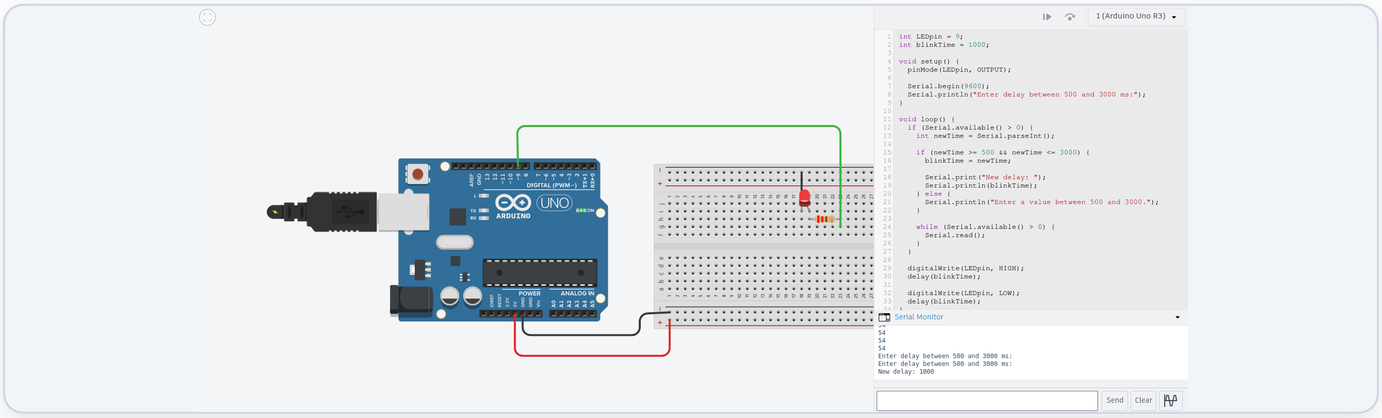
void loop() {
  if (Serial.available() > 0) {
    int newTime = Serial.parseInt();
    if (newTime >= 500 && newTime <= 3000) {
      blinkTime = newTime;
      Serial.print("New delay: ");
      Serial.println(blinkTime);
    } else {
      Serial.println("Enter a value between 500 and 3000.");
    }
    while (Serial.available() > 0) {
      Serial.read();
    }
  }
  digitalWrite(LEDpin, HIGH);
  delay(blinkTime);
  digitalWrite(LEDpin, LOW);
  delay(blinkTime);
}
```

نکات کلیدی

- اعتبارسنجی ورودی (newTime >= 500 && newTime <= 3000) از مقادیر نامعتبر جلوگیری می‌کند.
- پیام بازخورد در Serial Monitor به کاربر کمک می‌کند تا مقدار صحیح را وارد کند.

توضیح عملکرد

این مسئله نسخهٔ پیشرفته‌تر و «ایمن» مسئلهٔ ۲ است؛ کاربر سرعت چشمک‌زدن LED را از طریق سریال تعیین می‌کند، اما مقدار واردشده باید در بازهٔ مجاز ۵۰۰ تا ۳۰۰ میلی‌ثانیه باشد. اگر عدد در این بازه قرار گیرد، در blinkTime ذخیره و پیام تأیید چاپ می‌شود؛ در غیر این صورت پیام خطا نمایش داده می‌شود و مقدار پیشین حفظ می‌گردد. سپس LED با زمان تازه‌شده چشمک می‌زند. این اعتبارسنجی ورودی، مدار را در برابر مقادیر نامعتبر و رفتارهای غیرقابل‌پیش‌بینی مقاوم می‌سازد.



شکل ۱۱: محیط شبیه‌سازی Tinkercad Circuits — چیدمان بردبورد، سیم‌کشی و کد اجراشده

مسئلهٔ ۱۲ سیستم کنترل با چند دکمه و چند LED

صورت مسئله

سه دکمه به آردوینو متصل کنید؛ به‌گونه‌ای که با فشردن هر دکمه، یک LED متفاوت روشن شود (دکمهٔ اول LED قرمز، دکمهٔ دوم LED سبز و دکمهٔ سوم LED آبی).

کد آردوینو

```
int redButton = 2;
int greenButton = 3;
int blueButton = 4;
int redLED = 8;
int greenLED = 9;
int blueLED = 10;

void setup() {
  pinMode(redButton, INPUT_PULLUP);
  pinMode(greenButton, INPUT_PULLUP);
  pinMode(blueButton, INPUT_PULLUP);

  pinMode(redLED, OUTPUT);
  pinMode(greenLED, OUTPUT);
  pinMode(blueLED, OUTPUT);
}

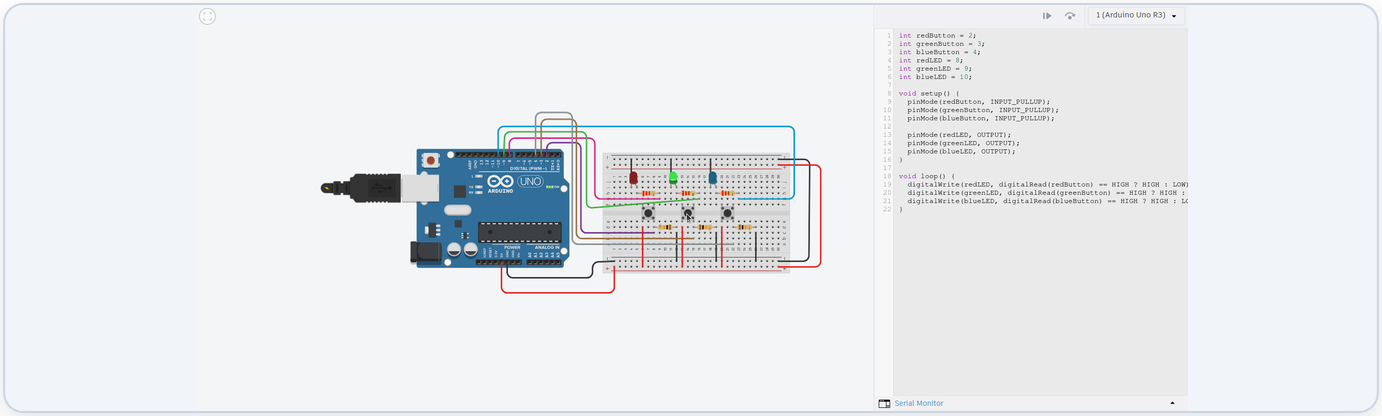
void loop() {
  digitalWrite(redLED, digitalRead(redButton) == LOW ? HIGH : LOW);
  digitalWrite(greenLED, digitalRead(greenButton) == LOW ? HIGH : LOW);
  digitalWrite(blueLED, digitalRead(blueButton) == LOW ? HIGH : LOW);
}
```

نکات کلیدی

- هر دکمه به‌صورت مستقل یک LED را کنترل می‌کند و ساختار ورودی/خروجی یک‌به‌یک حفظ شده است.
- با INPUT_PULLUP، فشردن دکمه خط را به زمین وصل کرده و مقدار LOW تولید می‌کند؛ بنابراین منطق معکوس شده است.

توضیح عملکرد

در این پروژه یک سامانهٔ کنترل چندکاناله ساخته شده است: سه دکمهٔ ورودی و سه LED خروجی، به‌گونه‌ای که هر دکمه به‌طور مستقل یک LED را کنترل می‌کند (قرمز، سبز و آبی). همهٔ پین‌های دکمه با INPUT_PULLUP پیکربندی شده‌اند تا نیازی به مقاومت خارجی نباشد؛ در این حالت فشردن دکمه خط را به زمین وصل کرده و مقدار LOW تولید می‌کند، بنابراین منطق شرطی به‌صورت معکوس نوشته شده است. این ساختار، پایهٔ طراحی صفحه‌کلیدها و پنل‌های کنترل صنعتی محسوب می‌شود.



شکل ۱۲: محیط شبیه‌سازی Tinkercad Circuits — چیدمان بردبورد، سیم‌کشی و کد اجراشده

۷. نتایج شبیه‌سازی

پس از اجرای هر دوازده مسئله در محیط Tinkercad، رفتار مشاهده‌شده با انتظار منطقی هر مدار مطابقت داشت. خلاصهٔ نتایج در جدول زیر آمده است:

مسئله	خروجی مورد انتظار	نتیجهٔ مشاهده‌شده	وضعیت
۱	چشمک LED هر یک ثانیه	روشن/خاموش منظم	✔ صحیح
۲	تغییر زمان با ورودی سریال	اعمال مقادیر واردشده	✔ صحیح
۳ و ۴	کنترل LED با دکمه / Toggle	پاسخ درست به فشردن	✔ صحیح
۵ و ۶	چشمک متناوب دو LED / زمان ۲ ثانیه	حالت مکمل و زمان‌بندی درست	✔ صحیح
۷ و ۸	ترتیب رنگ‌های RGB و چشمک دو LED	قرمز/سبز/آبی به‌درستی	✔ صحیح
۹ و ۱۰	واکنش به دما و نور	سوییچ در آستانهٔ ۳۰ درجه و مقدار نور	✔ صحیح
۱۱ و ۱۲	سرعت متغیر / سه دکمه و سه LED	اعتبارسنجی ورودی و کنترل مستقل	✔ صحیح

۸. تحلیل نتایج

در مدارهای کنترلی، LED هنگامی روشن می‌شود که پین خروجی در وضعیت `HIGH` (حدود ۵ ولت) قرار گیرد. در سنسورها، مقدار آنالوگ خوانده‌شده توسط `analogRead` به یک کمیت فیزیک تبدیل و با یک آستانه مقایسه می‌شود؛ این همان منطق تصمیم‌گیری دیجیتال است. استفاده از `INPUT_PULLUP` در مسائل ۴ و ۱۲ باعث شد منطق دکمه معکوس شود و نیازی به مقاومت خارجی نباشد.

۹. نتیجه‌گیری

این آزمایش نشان داد که میکروکنترلر آردوینو ابزاری قدرتمند و انعطاف‌پذیر برای پیاده‌سازی مدارهای دیجیتال است. با ترکیب فرمان‌های سادهٔ ورودی/خروجی دیجیتال و آنالوگ، توانستیم رفتارهای متنوعی از جمله چشمک‌زن، کنترل با دکمه، نمایشگر RGB و سامانهٔ هشدار دما و نور را بسازیم. استفاده از متغیرها و پورت سریال نیز امکان ساخت مدارهای پویا و قابل تنظیم را فراهم می‌کند.

۱۰. پیشنهادات

- استفاده از فرمان (PWM) `analogWrite` برای کنترل شدت روشنایی LED به‌جای فقط روشن و خاموش کردن.
- افزودن بازر (Buzzer) برای تبدیل هشدارهای نوری به هشدارهای صوتی در سنسور دما.
- پیاده‌سازی منطق ضدپرش (Debounce) پیشرفته‌تر با تابع `millis()` به‌جای `delay()` .
- طراحی گیت‌های منطقی AND، OR و NAND با ترکیب چند ورودی دیجیتال.

۱۱. منابع

1. جزوهٔ درس آزمایشگاه سیستم‌های دیجیتال، بخش آردوینو — م.ح. سلیمانی الموتی.
2. مستندات رسمی Arduino.cc — Reference: digitalWrite, pinMode, analogRead.
3. مستندات Autodesk Tinkercad Circuits.
4. Margolis, J. *Arduino Cookbook*, O'Reilly Media.

✦ پایان ✦